COMPUTER SCIENCE + COMPUTER ENGINEERING FOR GAMES

09

VOLUME NINE

PACKAGE 10 / GRADUATE SPECIALIZATION

Graduate numerical methods and research practice

Standalone teaching module; use the workbook and separate answer key

MODULE 41 / FOUR LESSONS

Edition: September 28, 2026

Original lesson sequence preserved. Self-study materials and runnable educational references; not an accredited degree or completed learner submission.

Contents

Module 41 — Graduate numerical methods and research practice 3

Lesson 41.1 — Reading research as claims, assumptions and evidence 4

Lesson 41.2 — Conditioning, sparse systems and iterative solvers 6

Lesson 41.3 — Constrained optimization, derivatives and optimality conditions 8

Lesson 41.4 — Experimental design, uncertainty and reproducible graduate work 10

Module 41 project: numerical research notebook 12

Primary reading directory 13

Module 41 — Graduate numerical methods and research practice

Prerequisites: Gates B, F and H; rigorous linear algebra, calculus, probability and systems competence.

Original project brief: Produce a graduate numerical notebook with derivations, reference implementations, conditioning experiments and a preregistered evaluation plan. Complete substantial assigned problem sets; the short lesson quizzes are only formative checks.

Core reference IDs: R42, R54, R09, R23. Full entries are in the reading directory.

- 41.1 — Reading research as claims, assumptions and evidence
- 41.2 — Conditioning, sparse systems and iterative solvers
- 41.3 — Constrained optimization, derivatives and optimality conditions
- 41.4 — Experimental design, uncertainty and reproducible graduate work

Lesson 41.1 — Reading research as claims, assumptions and evidence

Learning outcome: Extract an implementable argument rather than only a paper summary.

Planning: 18-26 hours. Includes assigned study and practical work; module/Gate I time overlaps.

Original teaching primer

A research paper makes claims under assumptions using a method and evidence. First identify the problem, baseline, objective and scope. Then reconstruct the central derivation or algorithm without looking at the authors' code. Track definitions, units and where approximations enter. A result can be valuable while narrower than its abstract initially suggests. Build a claim-evidence table: each claim points to an equation, proof, experiment or stated limitation. Distinguish reproduction of reported results from an independent implementation and from a new extension. Read related work to identify the actual contribution. Select original papers and author artifacts where available, and record versions. A negative reproduction result needs diagnostic investigation before concluding the original claim is false.

Turn a paper into an engineering contract

Write the question as an input-output relationship, not a topic: "For this scene family and error threshold, does method A reach the threshold in less total time than baseline B?" Specify scene motion, resolution, light/material model, numerical precision and the reference used to estimate error. The claim is a predicate over those conditions. An attractive image is an observation; it is not yet the predicate's proof. A reproducible implementation needs enough definitions that a second engineer can construct the same input and decide whether the output passes.

Separate three objects in your notes. The mathematical method defines ideal behavior. The implementation chooses finite precision, data layouts and scheduling. The experiment samples particular workloads on particular machines. A failure in one object does not automatically falsify the other two. For example, an unbiased estimator can be implemented with a wrong density; an accurate implementation can be compared with the wrong quality target. The assigned original papers supply the methods; this contract structure is the course's review procedure. [R44, S05]

Reconstruct before consulting implementation details

On the first pass, collect symbols, domains and units. On the second, derive the central update using a case small enough to calculate by hand. On the third, identify each approximation and each missing engineering choice. Do not hide an omitted detail with a convenient default: record it as an ambiguity and test whether plausible choices change the result. Only then compare your interpretation with author code, supplements or errata.

A claim-evidence map should record the exact figure, equation or artifact supporting each claim; the controlled comparison; one assumption that could break it; and your proposed test. Keep the original claim beside your narrower reproduction claim. This prevents a one-dimensional compliance experiment from becoming a claim to have reproduced a complete cloth benchmark.

Choose a tractable graduate contribution

A useful extension changes one dimension: a new workload distribution, a stricter correctness contract, a memory budget, a failure mode or an algorithmic component. Merely porting code is a contribution to an artifact, but does not by itself demonstrate a new scientific result. Conversely, a rigorous negative result can be valuable when it exposes a boundary and eliminates implementation error as the explanation.

Declare which stage is complete: method reconstruction, result reproduction, extension evaluation or independent review. These labels are deliberately explicit because communities use "reproducibility" and "replicability" differently. Passing the bundled smoke tests establishes none of the later stages on its own. [S14]

Worked engineering example

Suppose a baseline has RMSE 0.020 at 4 ms and 0.010 at 16 ms. A candidate has RMSE 0.012 at 4 ms and 0.009 at 8 ms. At the predeclared threshold RMSE <= 0.010, the measured qualifying times are 16 and 8 ms, giving 2x speedup among these tested configurations. At equal 4 ms cost, the candidate has 40% lower RMSE instead. Neither statement establishes its time at exactly 0.010 without another measurement. Record the two claims separately and do not interpolate across a quality curve without stating the interpolation rule.

Original worked check

A method reported faster at equal image error is not necessarily faster at equal sample count. Reproducing the wrong comparison target tests a different claim.

Practical assignment

Analyze one paper associated with R23, R44 or R55. Pass: a two-page claim map, a reconstructed central algorithm, at least three assumptions and a feasible reproduction target are reviewed by another reader.

Full execution steps and acceptance checks are in LAB_WORKBOOK.pdf under this same lesson ID. Your warm-up file is `code/exercises/lesson_41_1.py`.

Reference starting point: templates/CLAIM_EVIDENCE_MAP.md; code/run_experiments.py

Assigned study

Original assignment: R23: Further Reading sections for a chosen rendering topic; R44 or R55: one named original paper; handbook research-reading template.

Focused expansion: R44: abstract, algorithm and evaluation of the 2020 ReSTIR paper; or R55 / S05: XPBD (2016), Sections 3-5. Read the chosen paper in full before finalizing the reproduction boundary. Use S14 for artifact-review vocabulary.

Check your understanding

Attempt all three original questions before opening the key. A lesson also requires its reading, lab and graduate problems.

1. Why distinguish equal-error from equal-sample comparisons?
2. What makes a claim-evidence table useful?
3. Does a failed reproduction immediately disprove a paper?

Lesson 41.2 — Conditioning, sparse systems and iterative solvers

Learning outcome: Choose a solver from mathematical properties and error requirements.

Planning: 28-40 hours. Includes assigned study and practical work; module/Gate I time overlaps.

Original teaching primer

A linear system Ax=b has numerical behavior governed partly by conditioning: sensitivity of the solution to changes in data. Residual r=b-Ax measures equation mismatch, but a small residual need not imply a small forward error for an ill-conditioned system. Scaling and problem formulation matter before choosing a faster kernel. Sparse storage exploits structure, while iterative methods use repeated matrix-vector operations. Conjugate gradient requires a symmetric positive-definite setting for its standard guarantees. Preconditioning changes the system seen by the iteration to improve convergence without changing the intended solution when correctly applied. State stopping criteria, norm, precision and maximum work. Test a known solution and increasingly difficult conditioning, not only one friendly matrix.

Residual, forward error and the condition of the question

Let x* solve Ax=*b and let x be your estimate. Define e=x*-x and r=b-Ax. Then Ae=r, so e=A^-1 r when A is invertible. Using a consistent induced norm, ||e|| <= ||A^-1|| ||r||. Combining ||b|| <= ||A|| ||x|| *yields the relative bound* ||*e*||/||*x*|| <= kappa(A) ||r||/||b|| for nonzero b. A residual threshold alone cannot give a useful forward-error guarantee when the condition number is large.

For A=diag(1,10^-6), b=(1,10^-6) and estimate x=(1,0), the residual norm is 10^-6, while the error norm is 1. A small equation mismatch hides a completely wrong second coordinate. The example separates a well-implemented solver from a well-conditioned model. Scaling coordinates, removing redundant constraints and improving boundary conditions may matter more than faster arithmetic.

From a quadratic to preconditioned conjugate gradient

For symmetric positive-definite A, minimize E(x)=0.5 x^T A x-b^T x. Its gradient is Ax-b, and positive curvature gives a unique minimizer. CG constructs search directions that are conjugate under A rather than merely orthogonal in Euclidean space. Standard PCG additionally assumes a fixed SPD preconditioner M. Apply z=M^-1 r; do not explicitly invert a large matrix. A positive diagonal gives the simple Jacobi choice. [S01]

The teaching implementation begins with x=0, r=b, z=M^-1 r and p=z. At each step set alpha=(r^T z)/(p^T A p), update x <- x+alpha p and r <- r-alpha A p, compute z_new=M^-1 r_new, and use beta=(r_new^T z_new)/(r_old^T z_old) to form p_new=z_new+beta p. A nonpositive denominator is a breakdown signal, not a complete SPD test. A nonsymmetric input may slip past that check and still invalidate the derivation.

Storage and stopping are part of the algorithm

Compressed sparse row storage holds row offsets, column indices and nonzero values. Validate offsets, dimensions and index ranges before multiplication. For a tridiagonal problem, sparse multiplication performs work proportional to nonzeros rather than the square of the dimension. The dense solver in this package is an independent oracle for small problems, not a recommended production sparse factorization.

Stop when the measured norm satisfies max(atol,rtol*||b||), or a declared work cap is reached. Recompute the true residual before reporting success because recursive residual updates accumulate error. Report convergence status separately from the solution vector. Your experiments should include a zero right-hand side, a known solution, a poorly scaled SPD family, and an intentionally unsupported matrix. A preconditioner that costs more than it saves in iterations is not automatically an improvement.

Worked engineering example

Take A=[[4,1],[1,3]], b=(6,7), x0=0. Then r0=p0=(6,7), A p0=(31,27), r0^T r0=85 and p0^T A p0=375. The first alpha is 85/375=17/75. Thus x1=(1.36,1.5866667), which is not yet the exact solution (1,2). Verify A(1,2)=(6,7). In exact arithmetic CG needs at most two steps here; in floating point the declared tolerance determines whether the measured result is accepted. For the diagonal family in the runner, exact Jacobi scaling makes the preconditioned system the identity, so that special case can converge in one step. It does not establish that Jacobi works well on general sparse systems.

Original worked check

If A=diag(1,10^-6), its 2-norm condition number is 10^6. A small perturbation in the second equation can produce a much larger relative change in its associated solution component.

Practical assignment

Implement a sparse matrix-vector product and conjugate gradient for a verified SPD system. Pass: derive the relevant assumptions, compare residual and forward error, and evaluate a simple preconditioner over several conditioning levels.

Full execution steps and acceptance checks are in LAB_WORKBOOK.pdf under this same lesson ID. Your warm-up file is `code/exercises/lesson_41_2.py`.

Reference starting point: code/gradlab/numerical.py; code/tests/test_numerical.py

Assigned study

Original assignment: R54: linear systems, positive definiteness and numerical methods; R07: orthogonality and eigenvalues; R42: numerical optimization background.

Focused expansion: R54: positive-definite equilibrium systems; S01: Shewchuk, Sections 3, 8, 11-12 and Appendix B3; R42: numerical linear algebra background. Re-derive the two-dimensional update without copying implementation code.

Check your understanding

Attempt all three original questions before opening the key. A lesson also requires its reading, lab and graduate problems.

1. What is the 2-norm condition number of diag(1,10^-6)?
2. Why can a small residual coexist with substantial solution error?
3. What matrix property is required by standard conjugate gradient?

Lesson 41.3 — Constrained optimization, derivatives and optimality conditions

Learning outcome: Derive and verify an optimization method for an engineering objective.

Planning: 26-38 hours. Includes assigned study and practical work; module/Gate I time overlaps.

Original teaching primer

Optimization requires a defined objective, feasible set and numerical tolerance. For differentiable unconstrained problems, stationary points have zero gradient but may be minima, maxima or saddles. Convexity provides stronger global conclusions under the appropriate assumptions. Equality and inequality constraints introduce Lagrange multipliers and Karush-Kuhn-Tucker conditions, whose necessity or sufficiency depends on regularity and convexity. For nonlinear least squares, residual Jacobians support Gauss-Newton-style approximations; poor scaling or rank deficiency can require damping or regularization. Check analytic or automatic derivatives against finite differences over sensible step sizes. Gradient checks themselves have truncation and roundoff error. Report convergence histories and failed initializations rather than only a final successful parameter set.

State the optimization problem before choosing an optimizer

A fitting problem needs an objective, a feasible set and a parameterization. A low loss can coexist with impossible joint angles, a negative material coefficient or an invalid energy model. For f(x) with inequalities g_i(x)<=0 and equalities h_j(x)=0, write L=f+sum(lambda_i g_i)+sum(nu_j h_j). KKT combines primal feasibility, lambda_i>=0, stationarity of L and lambda_i g_i=0. Necessity needs appropriate regularity conditions; in convex problems, a feasible KKT point provides a global optimum under the standard convex formulation. Do not promote every stationary point to a minimum. [S02]

For an equality constraint, the multiplier is not constrained to be positive. Its sign depends on the chosen equality convention. Changing x+y-1 to 1-x-y reverses the multiplier sign without changing the solution. Numerical checks should compare stationarity and feasibility, not only a memorized multiplier.

Derive a solver you can independently verify

Minimize 0.5||x-t||^2 subject to sum(x)=s. Stationarity gives x_i-t_i+lambda=0. Summing yields lambda=(sum(t)-s)/n, so x=t-lambda*1. This affine projection is the executable reference; it is not a general nonlinear constrained solver. With t=(0.2,0.9), s=1, lambda=0.05 and x=(0.15,0.85).

For nonlinear least squares 0.5||r(theta)||^2, the gradient is J^T r and the exact Hessian is J^T J+sum(r_i Hessian(r_i)). Gauss-Newton neglects the second term; damping solves a modified system such as (J^T J+mu I)delta=-J^T r. Poorly scaled parameters, rank deficiency and large residuals can make a naive step ineffective. A line search or trust region evaluates whether the step actually improves the intended objective.

Differentiate a game-relevant model, not just a toy polynomial

For a planar two-link arm, p(theta)=(l1 cos(t1)+l2 cos(t1+t2), l1 sin(t1)+l2 sin(t1+t2)). Its first Jacobian column differentiates both links, while the second differentiates only the second link. A straight arm can lose local ability to move in one Cartesian direction: the Jacobian becomes poorly conditioned. This links inverse kinematics to Lesson 41.2 instead of treating optimizer failure as magic.

Check each derivative using centered differences at several step sizes, preferably away from discontinuities. A tolerance should reflect scale: compare absolute error near zero and relative error when the derivative is substantial. Then deliberately test a singular pose, an unreachable target and a bound-active solution. Preserve the failure report. An optimizer that returns a status and feasible fallback is more useful than one that silently returns its last iterate.

Worked engineering example

For the manifest problem f=x^2+y^2, h=x+y-1, stationarity is 2x+lambda=0 and 2y+lambda=0. Subtracting gives x=y; feasibility gives x=y=0.5 and lambda=-1. The objective is 0.5. Along the feasible line y=1-x, f=2(x-0.5)^2+0.5, which independently proves the unique minimizer. For a centered derivative check of x^3 at x=2, [(2+h)^3-(2-h)^3]/(2h)=12+h^2 in exact arithmetic. This shows the truncation term explicitly; floating-point cancellation adds a different error as h becomes very small.

Original worked check

Minimize x^2+y^2 subject to x+y=1. The Lagrangian gives 2x+lambda=0 and 2y+lambda=0; together with the constraint, x=y=0.5 and lambda=-1.

Practical assignment

Derive the example, then solve a constrained fitting or IK problem with a numerical method. Pass: derivative checks, feasibility residual, convergence criteria and at least one failure or sensitivity case are reported.

Full execution steps and acceptance checks are in LAB_WORKBOOK.pdf under this same lesson ID. Your warm-up file is `code/exercises/lesson_41_3.py`.

Reference starting point: code/gradlab/numerical.py: constrained_fit, finite_gradient

Assigned study

Original assignment: R42: convexity, duality, optimality and numerical methods; R41: gradients and constrained differentials.

Focused expansion: R42 / S02: Convex Optimization, Sections 4.1, 5.5 and 9.1-9.5; R41: gradients and constrained differentials. Study constraint qualifications separately from convex sufficiency.

Check your understanding

Attempt all three original questions before opening the key. A lesson also requires its reading, lab and graduate problems.

1. What is the constrained minimizer in the example?
2. Does zero gradient prove a general nonconvex point is a minimum?
3. Why vary finite-difference step size in a derivative check?

Lesson 41.4 — Experimental design, uncertainty and reproducible graduate work

Learning outcome: Make a performance or quality claim that survives independent scrutiny.

Planning: 22-32 hours. Includes assigned study and practical work; module/Gate I time overlaps.

Original teaching primer

Define the primary outcome and comparison before examining final results. Choose workloads that represent the intended claim, plus adverse and held-out cases. Repeated frames within one run can be correlated, so the statistical unit might be a run, scene or device rather than every frame. A large count of correlated observations does not automatically provide strong independent evidence. Use confidence intervals or other justified uncertainty summaries, report effect sizes and preserve raw observations. Ablations remove or replace components to test which parts cause an improvement. Multiple comparisons and parameter tuning can overfit the benchmark suite. Keep final evaluation data separate from tuning when feasible. State external validity limits, including devices, workloads and quality settings not tested.

Define what an observation represents

The unit of inference is the object your claim generalizes over. Frames nested in one run share scene content, thermal history and scheduling behavior. Treating every frame as an independent experiment artificially increases apparent evidence. A study about scenes may need a scene-level summary; a study about repeated execution on one machine may use independent process runs. State the hierarchy explicitly: device, scene, run and frame. [R09]

Pair a baseline and candidate on the same workload when this removes irrelevant variation. Randomize execution order to reduce systematic warmup or drift effects. Keep the seed policy fixed and explain common-random-number pairing when it is used. A run killed by an unrelated system failure may be excluded under a predefined rule; a slow run caused by the method itself is a result, not an inconvenience.

Effect size, uncertainty and diagnostic evidence

For positive paired times a_i and b_i, log speedups z_i=log(a_i/b_i) have a useful additive scale. Report exp(mean(z_i)) when a geometric-mean speedup is the chosen estimand. It is not the same as sum(a_i)/sum(b_i), which answers a total-time question. The included bootstrap resamples whole pairs and returns percentile bounds. Its numerical output cannot establish independence, repair a biased workload suite or justify a sample size.

The bundled native benchmark is a within-process diagnostic, with 12 randomized-order pairs. Those rows demonstrate data capture and timer boundaries; they are not enough to claim broad CPU or GPU superiority. For an actual study, repeat independent processes, retain environment records and justify how uncertainty is calculated at the intended unit. State the confidence procedure rather than treating a 95% interval as a 95% probability statement about a fixed unknown parameter.

Ablations and sensitivity answer different questions

An ablation removes or replaces a component while preserving other controls. A sensitivity sweep changes a parameter to reveal the boundary of an effect. Both can be misleading when quality changes simultaneously with time. Compare a denoiser with and without history at equal total budget, and also disclose the resulting error; do not call a quality loss a pure optimization.

Reserve final scenes or workload families before tuning. Choose the main metric, analysis, exclusions and stopping rule before examining the final comparison. Keep secondary findings labeled exploratory. Publish raw rows, not only a plotted mean, and give the reviewer a command that regenerates the table from those rows. Failure to reproduce a timing exactly is not automatically a failure to reproduce the engineering conclusion; define acceptable numerical and statistical agreement in advance.

Worked engineering example

Two paired workloads take baseline times (10,100) ms and candidate times (5,90) ms. The geometric-mean speedup is sqrt(2*(100/90)), approximately 1.491x. The total-time speedup is 110/95, approximately 1.158x. Both calculations are correct but weight the workloads differently. A presentation that switches between them after seeing the data changes the question. If each value was itself the mean of 1000 correlated frames, those 4000 frames do not automatically provide 4000 independent speedup observations.

Original worked check

Ten thousand frames from one scene are not equivalent to ten thousand independent games. A scene-level comparison across diverse scenes can better test a claim about broad engine behavior.

Practical assignment

Preregister a small experiment with baseline, primary metric, trial unit and exclusion policy. Pass: raw data and analysis scripts reproduce the tables, and an ablation isolates at least one proposed mechanism.

Full execution steps and acceptance checks are in LAB_WORKBOOK.pdf under this same lesson ID. Your warm-up file is `code/exercises/lesson_41_4.py`.

Reference starting point: code/gradlab/numerical.py: paired_bootstrap; evidence/reference_run/native_scan_pairs.csv

Assigned study

Original assignment: R09: statistical inference; R42: experimental examples; handbook benchmark and research templates.

Focused expansion: R09: inference, confidence intervals and experimental units; S14: artifact evaluation. Use the package PREREGISTRATION and BENCHMARK_MANIFEST templates before collecting final data.

Check your understanding

Attempt all three original questions before opening the key. A lesson also requires its reading, lab and graduate problems.

1. Why identify the statistical unit?
2. What does an ablation test?
3. Why reserve held-out workloads?

Module 41 project: numerical research notebook

Produce a notebook or script-driven report linking one research claim to a derivation, an independent small oracle and a preregistered evaluation. Implement CSR matrix-vector multiplication and PCG for a verified SPD family. Include at least one non-diagonal family; a diagonal Jacobi example alone is not enough. Compare forward error, true residual, work, setup and solve cost. Add a constrained fitting/IK problem with derivative checks, feasibility and a failure case.

Required artifacts: claim map; equations and units; generated test matrices with known solutions; derivative-step sweep; raw solver histories; preregistration; analysis script; a short reviewer response. A reasonable staged plan is mathematical contract, independent oracle, implementation, then experiments. Correctness 35 points, derivation 25, experimental design 25, reproducibility/limitations 15. Suggested pass: 80/100 with no unexplained numerical mismatch or missing assumption.

Integration checkpoint: explain how solver conditioning or research comparison choices will affect your proposed Gate I track. Do not infer a broad performance conclusion from the included eight-row diagonal diagnostic.

Primary reading directory

R01-R55 retain the original program IDs. S01-S17 are focused additions for this expansion. Citations such as [S05] refer to the entries below; equations and worked cases not attributed to another source are original teaching derivations. External resources retain their own rights and are linked rather than redistributed. Moving documentation is not a promise of compatibility with an untested installed version. Record exact versions for your own artifact.

R07 — 18.06 Linear Algebra

MIT OpenCourseWare

https://ocw.mit.edu/courses/18-06-linear-algebra-spring-2010/

Matrices, vector spaces, eigenvalues and problem sets.

Source status: Inherited from the original program; not every moving page was revalidated for this expansion.

R09 — 18.05 Introduction to Probability and Statistics

MIT OpenCourseWare

https://ocw.mit.edu/courses/18-05-introduction-to-probability-and-statistics-spring-2022/

Probability, inference and quantitative reasoning.

Source status: Inherited from the original program; not every moving page was revalidated for this expansion.

R23 — Physically Based Rendering, fourth edition

Matt Pharr, Wenzel Jakob, Greg Humphreys

https://www.pbr-book.org/4ed/contents

Rendering mathematics and implementation; use chapter titles in lesson assignments.

Source status: Inherited from the original program; not every moving page was revalidated for this expansion.

R41 — 18.02SC Multivariable Calculus

MIT OpenCourseWare

https://ocw.mit.edu/courses/18-02sc-multivariable-calculus-fall-2010/

Partial derivatives, gradients, multiple integrals and constrained differentials.

Source status: Inherited from the original program; not every moving page was revalidated for this expansion.

R42 — Convex Optimization

Stephen Boyd and Lieven Vandenberghe

https://web.stanford.edu/~boyd/cvxbook/

Graduate optimization text available from its authors; copyrighted, linked rather than redistributed.

Source status: Inherited from the original program; not every moving page was revalidated for this expansion.

R44 — Spatiotemporal reservoir resampling for real-time ray tracing with dynamic direct lighting

Benedikt Bitterli and coauthors

https://research.nvidia.com/publication/2020-07_spatiotemporal-reservoir-resampling-real-time-ray-tracing-dynamic-direct

Original ReSTIR research, 2020; example reproduction target, not a claim to be the latest method.

Source status: Inherited from the original program; not every moving page was revalidated for this expansion.

R54 — 18.085 Computational Science and Engineering I

MIT OpenCourseWare

https://ocw.mit.edu/courses/18-085-computational-science-and-engineering-i-fall-2008/

Graduate numerical methods, equilibrium systems, differential equations and variational reasoning.

Source status: Inherited from the original program; not every moving page was revalidated for this expansion.

R55 — Author publication archive: position-based simulation and XPBD

Matthias Mueller and collaborators

https://matthias-research.github.io/pages/publications/publications.html

Read the named 2016 XPBD paper and related course notes for the simulation reproduction; papers retain their own copyrights.

Source status: Inherited from the original program; not every moving page was revalidated for this expansion.

S01 — An Introduction to the Conjugate Gradient Method Without the Agonizing Pain (1994)

Jonathan Richard Shewchuk

https://www.cs.cmu.edu/~quake-papers/painless-conjugate-gradient.pdf

PCG derivation and Appendix B3; PDF and algorithm page inspected.

Source status: Focused primary source consulted September 28, 2026; see note for retrieval limits.

S02 — Convex Optimization (2004), author-hosted text

Stephen Boyd and Lieven Vandenberghe

https://web.stanford.edu/~boyd/cvxbook/bv_cvxbook.pdf

Convexity, optimality and numerical methods; KKT page inspected. Also R42.

Source status: Focused primary source consulted September 28, 2026; see note for retrieval limits.

S05 — XPBD: Position-Based Simulation of Compliant Constrained Dynamics (2016)

Miles Macklin, Matthias Mueller and Nuttapong Chentanez

https://matthias-research.github.io/pages/publications/XPBD.pdf

Original paper; multiplier/compliance equations inspected. A bounded reproduction target, not a completed result supplied here.

Source status: Focused primary source consulted September 28, 2026; see note for retrieval limits.

S14 — Artifact Review and Badging, version 1.1

ACM

https://www.acm.org/publications/policies/artifact-review-and-badging-current

Official search index and ACM terminology page verified; direct policy-page fetch returned 403. Supplement: https://www.acm.org/publications/badging-terms . This curriculum awards no ACM badge.

Source status: Focused primary source consulted September 28, 2026; see note for retrieval limits.